

Architecture note — Media Monitoring Agent

Problem. Six analysts spend 3+ hours each morning reading ~40 outlets to brief the Director General by 07:30. Quality varies by shift, items get missed, and negative stories breaking outside the morning cycle go unnoticed for hours.

Solution in one line. A pipeline reads the coverage, tags it, drafts a cited briefing, and hands it to a human to approve. The analysts stay in charge; the machine does the reading.

Components

Component	Role	Why
Python (agent/)	Ingest, de-duplicate, classify, draft, check	Plain functions; each step runs and tests alone
Claude Haiku 4.5	Classify each item: theme, sentiment, priority, reason	1,500 small decisions/day: speed and cost dominate
Claude Sonnet 5.5	Write the briefing; answer archive questions	One document/day read by the DG: quality dominates
SQLite	Archive of articles and briefings, inside the client's environment	One file, no server; Postgres later is a small change
FastAPI (api.py)	Nine endpoints n8n calls; API-key auth; clean JSON errors	Adds no logic; the brain stays in agent/
n8n	Scheduling, human approval, delivery, alert cadence	The required backbone; the client's team can change the flow without code

LLM choice and alternatives: see docs/llm-choice.md. agent/llm.py is the single swap point.

Data flow

06:00 Schedule → POST /pipeline/daily → draft emailed to analyst → WAIT (form)
approved → record → HTML email to DG office
rejected → record → notify analyst
every 10 min Schedule → ingest → classify → new high-risk? → ONE alert email → acknowledge

How the Director can trust what reaches the DG's desk

1. **Nothing is sent without a named human approving it**; decision, name and time are stored.
2. **Every claim carries a citation**, and code verifies each [n] maps to a supplied article; a made-up one is flagged at the top of the draft.
3. **The model may use only the supplied items** — and in practice declines to link stories without evidence.
4. **The high-risk rule is code, not learned**: negative + high priority + a sector theme. The client can read and change it.
5. **Measured, not assumed**: 83% theme / 80% sentiment agreement with a human labeller on a blind sample, disagreements listed. Boundary cases are why approval is mandatory.
6. **Failures are visible, never silent**: dead feeds, truncated drafts and bad citations become warnings on the draft; a failing step stops the workflow instead of emailing nothing.

Security and data residency

n8n and the database run inside the client's environment; the only outbound traffic is article text to the LLM API over HTTPS. Secrets live in .env, never in code or Git, and every API call needs a key. Production adds HTTPS in front of n8n and the API, role-based access (analysts approve, DG office reads), a regional LLM endpoint if policy requires data to stay in-Kingdom, and an audit trail from the briefings table plus n8n execution history.

Cost (1,500 items/day)

≈ \$33/month in LLM calls (classification ≈ \$31 measured, briefing ≈ \$2), ≈ \$19 with prompt caching, plus a small VM for n8n. Against: 3 hours × 6 analysts every morning.

Scope cuts (deliberate)

Arabic sources and WhatsApp/Teams delivery. The core loop runs end to end with both approval branches tested, and archive Q&A is built in its simplest grounded form.

What's next

1. Feedback loop from approvals: measurement, a growing labelled set, and reviewed prompt examples. Nothing learns automatically.
2. Social media, the same way as news: public posts about Saudi tourism on X, Instagram and TikTok, read through each platform's official API or a licensed listening feed (as news is read through RSS — nothing scraped), with a reach-aware risk rule.
3. Arabic sources and a bilingual codebook.
4. Client-chosen LLM provider behind agent/llm.py, verified against the evaluation set.
5. Teams, Slack and WhatsApp delivery as n8n nodes on the approved branch.
6. Embedding retrieval for the archive Q&A.